A Bash Programming Introduction Tutorial

Operating Systems

Universität Innsbruck

EdgeAI Group

Summer Semester 2026

1 Introduction to shell

A shell is an interactive command interpreter and also a programming language. We will first start by simply calling commands and later turn to the programming/scripting features of the shell. There exist different types of shells, like C-shell, Bash, zsh, etc. In this course, we will use the most commonly used shell: Bourne Again SHell (Bash).

Running commands

From the interactive command line we can start a command by typing:

```
command-name argument-0 argument-1 ...
```

and then pressing enter. For example:

```
ls --help
```

will call the command `ls` and the `--help` argument will make `ls` output documentation on how to use the command.

echo and arguments

Let's now turn to the useful command `echo`, which we will use a lot. This command will print its arguments to stdout. For example:

```
echo hello world
```

will print:

```
hello world
```

Now lets add some spaces between hello and world:

```
echo hello           world
```

which gives:

```
hello world
```

The reason why the additional spaces disappear, is that "hello" and "world" are two separate arguments given to the `echo` command. White spaces between arguments are ignored. In order to print our string with the additional spaces, we can surround it with quotation marks (`"`):

```
echo "hello           world"
```

which gives:

```
hello           world
```

Note that, unlike before, where `echo` received two arguments, it now receives one argument because of the quotation marks.

We can prevent spaces being interpreted as argument separators by escaping them with \:

```
echo hello\ \ world
```

which gives:

```
hello  world
```

Again, in this example `echo` received only one argument.

Running multiple commands sequentially

Usually, we run one command, or later, when we create scripts, run one command per line. However, we can also run multiple commands sequentially from one line by separating them with a semicolon (`;`). For example:

```
echo hello; echo world
```

which gives:

```
hello
world
```

Running a command in the background

When you normally run a command from the command line, its process runs in the foreground, meaning that what you type is written to its standard input (stdin), and we cannot run any new commands until the process terminates. However, it is possible to run a command in the background by adding `&` after the command, which decouples stdin from your terminal, allowing us to continue running other commands while the background process continues. For example:

```
sleep 30 &
```

2 Common Commands

Directory navigation and paths

Show your current working directory:

```
pwd
```

output:

```
/home/alice/
```

change to subdirectory:

```
cd demo
```

and run `pwd` again:

```
cd demo
```

output:

```
/home/alice/demo
```

When specifying paths, we can use absolute paths or paths relative to the current working directory. If the path starts with `/`, it is treated as an absolute path:

```
cd /home/alice/demo
```

Otherwise, it is relative to the current working directory.

Note that each directory path "contains" two virtual directories `.` and `..` which correspond to the directory itself and its parent directory respectively. For example:

```
cd /home/alice/./demo
```

is the same as:

```
cd /home/alice/demo
```

and as

```
cd /home/alice/../alice/demo
```

Now the latter one is usefull when we want to navigate to the parent directory of our current working directory:

```
cd ..
pwd
```

output:

```
/home/alice
```

Let's go back to the demo directory:

```
cd demo
```

We can use the command `ls` to list the contents of a directory. Without any arguments, it will list the contents of the current working directory:

```
ls
```

output:

```
my_files  some_text_file.txt  tmp
```

With argument `-l` it will show us some more details:

```
total 12
drwxr-xr-x 2 alice alice 4096 Jan 23 14:40 my_files
-rw-r--r-- 1 alice alice 3972 Jan 23 14:42 some_text_file.txt
drwxr-xr-x 2 alice alice 4096 Jan 23 14:41 tmp
```

Here we can see (from the `d` at the beginning of each line) whether it's a directory or a file, the number of links to the file/directory, the user that owns the file, the group that owns the file, its size, modification date, and name.

We can also pass the path of a directory as an argument to see its contents:

```
ls my_files
```

output

```
another_file.txt  some_file.txt
```

We can also use the asterisk (`*`) as a wild card when specifying paths as an argument of a command. For example, to match all files/directories that contain the word `file` in the current working directory, we would write `./*file*`. A command's argument with a wild card will be expanded such that each file/directory that matches the path with the wild card, appears as a separate argument. We can easily illustrate this using the `echo` command because it prints all its arguments to the standard output (stdout):

```
echo ./*file*
```

output:

```
./my_files ./some_text_file.txt
```

These are the two files that match the path `./*file*`, and they have been used as the two arguments of `echo`. That is, the above call to `echo` was equivalent to calling

```
echo ./my_files ./some_text_file.txt
```

File creation and deletion

We can create a directory using `mkdir` (followed by verifying it's creation using (ls)):

```
mkdir mydir
ls -l
```

output

```
total 12
drwxr-xr-x 2 alice alice 4096 Jan 23 14:40 my_files
drwxr-xr-x 2 alice alice 4096 Jan 23 16:59 mydir
-rw-r--r-- 1 alice alice 3972 Jan 23 14:42 some_text_file.txt
drwxr-xr-x 2 alice alice 4096 Jan 23 14:41 tmp
```

We can remove a directory with `rmdir`:

```
rmdir mydir
ls -l
```

output:

```
total 12
drwxr-xr-x 2 alice alice 4096 Jan 23 14:40 my_files
-rw-r--r-- 1 alice alice 3972 Jan 23 14:42 some_text_file.txt
drwxr-xr-x 2 alice alice 4096 Jan 23 14:41 tmp
```

The command `touch` can be used to update a file's modification date or to create a file if it does not exist. Let's use it for the latter case:

```
touch myfile
ls -l
```

output:

```
total 12
drwxr-xr-x 2 alice alice 4096 Jan 23 14:40 my_files
-rw-r--r-- 1 alice alice 0    Jan 23 17:05 myfile
-rw-r--r-- 1 alice alice 3972 Jan 23 14:42 some_text_file.txt
drwxr-xr-x 2 alice alice 4096 Jan 23 14:41 tmp
```

We can remove a file using `rm`:

```
rm myfile
ls -l
```

output:

```
total 12
drwxr-xr-x 2 alice alice 4096 Jan 23 14:40 my_files
-rw-r--r-- 1 alice alice 3972 Jan 23 14:42 some_text_file.txt
drwxr-xr-x 2 alice alice 4096 Jan 23 14:41 tmp
```

Moving and copying files

With the command `mv` we can move files/directories. Let's move `my_file/some_file.txt` to the current working directory:

```
mv my_files/some_file.txt .
ls -l
```

output:

```
total 12
drwxr-xr-x 2 alice alice 4096 Jan 23 14:40 my_files
-rw-r--r-- 1 alice alice    0 Jan 23 16:53 some_file.txt
-rw-r--r-- 1 alice alice 3972 Jan 23 14:42 some_text_file.txt
drwxr-xr-x 2 alice alice 4096 Jan 23 14:41 tmp
```

Note that the second argument, the destination, can be an existing directory path (like in the example above) or can be an existing directory path plus a new file/directory name. In such a case, the file/directory is not only moved to a different directory of also given a new name:

```
mv some_file.txt my_files/some_file_with_a_new_name.txt
ls my_files
```

output:

```
another_file.txt  some_file_with_a_new_name.txt
```

Note that we can also only rename a file without moving it. For example:

```
mv some_text_file.txt a_text_file.txt
ls -l
```

output:

```
total 12
drwxr-xr-x 2 alice alice 4096 Jan 23 14:40 my_files
-rw-r--r-- 1 alice alice 3972 Jan 23 14:42 a_text_file.txt
drwxr-xr-x 2 alice alice 4096 Jan 23 14:41 tmp
```

We can copy files/directories with the `cp` command in a syntactically similar way to `mv`:

```
cp a_text_file.txt a_copied_text_file.txt
ls -l
```

output:

```
total 16
-rw-r--r-- 1 alice alice 3972 Jan 23 17:21 a_copied_text_file.txt
-rw-r--r-- 1 alice alice 3972 Jan 23 14:42 a_text_file.txt
drwxr-xr-x 2 alice alice 4096 Jan 23 17:15 my_files
drwxr-xr-x 5 alice alice 4096 Jan 23 16:49 tmp
```

Note that when we want to copy a directory, we need to add the argument `-r` (r for recursive):

```
cp -r my_files my_files_copy
ls -l
```

output:

```
total 16
-rw-r--r-- 1 alice alice 3972 Jan 23 17:21 a_copied_text_file.txt
-rw-r--r-- 1 alice alice 3972 Jan 23 14:42 a_text_file.txt
drwxr-xr-x 2 alice alice 4096 Jan 23 17:15 my_files
drwxr-xr-x 2 alice alice 4096 Jan 23 17:25 my_files_copy
drwxr-xr-x 5 alice alice 4096 Jan 23 16:49 tmp
```

Reading text files

We can use the command `cat` to print the contents of a text file to the stdout:

```
cat a_text_file.txt
```

`cat` can accept multiple files as its arguments, in which case it will concatenate (hence the name) the contents of these files to the stdout:

```
cat a_text_file.txt a_copied_text_file.txt
```

The command `head` (or `tail`) can be used to only output the first (or last in case of `tail`) n lines (the number n should be specified after the `-n` argument) of a text file. For example to only show the first 2 lines of `a_text_file.txt`, we run:

```
head -n 2 a_text_file.txt
```

The command `grep` can be used to find occurrences of a string in a file. For example, to find the occurrences of the word "Duis" in `a_text_file.txt` and print all lines that contain this string, we run:

```
grep Duis a_text_file.txt
```

We can also use it to recursively search for a string in a directory, by specifying a directory name instead of a file name and adding the `-R` argument. For example, to search in the current working directory, we run:

```
grep Duis ./ -R
```

Note that if we run `grep` without specifying a file or directory name, it will read from stdin.

The `sed` command is a powerful tool for filtering and transforming text and supports regular expressions. We don't go into the details of this, but demonstrate how it can be used to print the contents of `a_text_file.txt` where all occurrences of "ipsum" are replaced by "pupsum":

```
sed "s/ipsum/pupsum/g" a_text_file.txt
```

Like with `grep`, `sed` will read from stdin if no file name is specified.

Manuals

The `man` command can be used to display the manual of a command. Let us, for example, have a look the the manual of `ls`:

```
man ls
```

You can use the arrow up/down, page up/down, home, and end keys of your keyboard to navigate through the manual. Press q to quit the program.

Man-pages are categorized by section numbers (from the `man` manual):

```
1   Executable programs or shell commands
2   System calls (functions provided by the kernel)
3   Library calls (functions within program libraries)
4   Special files (usually found in /dev)
5   File formats and conventions, e.g.,/etc/passwd
6   Games
7   Miscellaneous (including macro packages and conventions), e.g., man(7), groff(7), man-pages(7)
8   System administration commands (usually only for root)
9   Kernel routines [Non standard]
```

Now, it may be the case that multiple manual pages have the same name in different sections. For example, there is a manual page for the C-function `printf`, but also for the bash command `printf`. To see in which section there are manual pages available for a certain name, add the `-f` argument:

```
man -f printf
```

output:

```
printf (1) - Print output based on the format string and preceding arguments.
printf (3) - formatted output conversion
```

Now let's open the manual page for the bash command `printf` by providing the correct section number (1) in front of the name of the manual:

```
man 1 printf
```

File permission tools

The command `chmod` can be used to change file/directory permissions. The commands `chown` and `chgrp` can be used to change a file's/directory's owner and group, respectively. Have a look at the manual pages for these commands to learn how to use them.

Process tools

The command `ps` can be used to print a list of running processes. Without any arguments it prints all subprocesses of the current shell:

```
ps
```

output:

```
    PID TTY          TIME CMD
  15519 pts/2    00:00:00 bash
  34241 pts/2    00:00:00 ps
```

To see all processes of the current user run:

```
ps -ax
```

Now it is very hard to see parent-child relationships from this list. For this, the command `pstree` is very useful, which prints all processes as a tree:

```
pstree
```

Now let's start a second terminal where we run the following command:

```
sleep 1234567
```

The `sleep` command takes a number as an argument, waits this number of seconds, and then terminates. So the `sleep` process we just started will be running for a while.

Then go back to the first terminal and execute the following to find the process ID of the `sleep` process we just started:

```
ps -ax | grep "sleep 1234567"
```

This forwards the output of `ps` as input to `grep`, which searches for the line containing "sleep 1234567" (later, we will discuss the `|` operator in detail). It will give an output like this:

```
  34709 pts/1    S+     0:00 sleep 1234567
  34717 pts/2    S+     0:00 grep --color=auto sleep 1234567
```

The first line corresponds to the `sleep` command we started, while the second command corresponds to the process of `grep` itself, which also shows up here because the string we are looking for appears in its arguments. The first line tells us the process ID that we were looking for (34709 in this example).

Let's now kill this process using the `kill` command to send the `SIGKILL` to it (replace the process ID by the ID you found):

```
kill -SIGKILL 34709
```

Now let's check whether the `sleep` process is still running:

```
ps -ax | grep "sleep 1234567"
```

output:

```
  34817 pts/2    S+     0:00 grep --color=auto sleep 1234567
```

3 Keyboard shortcuts

Ctrl+C

This key combination terminates the currently running (foreground) process. Let's try it! Run:

```
sleep 100
```

Now press the Ctrl+C key combination. You will see that sleep was terminated and you can run a new command again.

Ctrl+Z

This key combination suspends the currently running (foreground) process. Let's try it! Run:

```
sleep 100
```

Now press the Ctrl+Z key combination. You will see

```
^Z
[1]+  Stopped                 sleep 100
```

and can run commands again. Now we can continue the execution of sleep by using the `fg` command, which continues the execution of the command and brings it to the foreground (hence the name) again:

```
fg
```

Now the execution of the `sleep` command resumes.

Instead of resuming a process in the foreground, we can also resume it in the background with the command `bg`. When a process runs in the background, its stdin is detached, allowing you to continue running other commands.

Tab completion

We can use the Tab key to automatically complete commands or paths. For example, let's try to complete the `echo` command name with tab. Type (without hitting enter):

```
ech
```

and now hit Tab. Now the command name will be autocompleted to:

```
echo
```

As mentioned before, we can also use tab completion for path names. For example, type (without hitting enter):

```
cat a_t
```

and now hit Tab. Now the path will be autocompleted to:

```
cat a_text_file.txt
```

If there are more ways in which the currently typed command or path can be completed, then a list of all options will be printed. For example type (without hitting enter):

```
cat my_files/
```

and now hit Tab. Since there are multiple files in the `my_files` directory, there a multiple ways to complete the path, so all options are printed:

```
another_file.txt  some_file_with_a_new_name.txt
```

4 Variables

4.1 Assigning values to variables

Let us demonstrate how to (instantiate and) assign a value to a variable, in this case variable `val`:

```
val=foo
```

It is important to note here that there must be no space between neither the name of the variable and the `=` operator, nor the `=` operator and the value. If the value contains spaces, then we must either surround it with quotation marks (`"`) or escape the white space:

```
val=foo\ bar
```

Using variables

We can get the value of a variable by adding the `$` sign in front of it. For example:

```
echo $val
```

output:

```
foo bar
```

Arrays

We can also use arrays in Bash. Let's create an array the the elements `alice`, `bob` and `charlie`:

```
names=(alice bob charlie)
```

Let's print the element of this array with index 1:

```
echo ${names[1]}
```

output:

```
bob
```

With index `@` (standing for "all") we can get all elements of the array:

```
echo ${names[@]}
```

output:

```
alice bob charlie
```

Environment variables

Environment variables are variables that a child process inherits from it's parent. They can be useful for passing on system configuration information. Their values can be accessed in the same way as normal variables. For example, the `HOME` environment variable contains the path of the user's home directory:

```
echo $HOME
```

output:

```
/home/alice
```

Now also let's make an environment variable ourselves:

```
myenv=test
```

So far `myenv` is only a normal variable, but we can turn it into an environment variable by running:

```
export myenv
```

Now let's verify whether it is really an environment variable. We do this by creating a child process. Note that `bash` is a command like any other. So we can also call it from within an existing bash session:

```
bash
```

Now our original `bash` process created a new `bash` process as its child and this child is currently active. So let's see whether our environment variable was passed on to this child process:

```
echo $myenv
```

output:

```
test
```

Now let's also see whether the previously created variable `val` was passed on:

```
echo $val
```

but now the output is an empty line, which is what we would expect because `val` is not an environment variable that was passed on the the new process. Let's terminate the current `bash` process by calling:

```
exit
```

Now we are back again at the original (parent) `bash` process. So now let's verify that variable `val` still exists here:

```
echo $val
```

output:

```
foo bar
```

Special variables

Bash provides special variables that reflect process state and arguments. For example

- `$?`: exit status of the previous command
- `$!`: process ID of the last background process
- `$$`: process ID of the current process
- `$0`: the name of the current shell/script
- `$1`, `$2`, `$3`,...: the arguments of the current shell/script
- `$#`: the number of arguments of the current shell/script

5 Arithmetic

We can do arithmetic by writing `$((expression))`, where `expression` is the arithmetic expression to be evaluated and can contain variables as operands. In an expression, variable names must not be preceded by the `$` symbol. The operators and their precedence, associativity, and values are the same as in the C language.

Examples:

```
x=7
y=3
echo $((x + y))
echo $((x * y))
echo $((x / y))
echo $((x % y))
```

output:

```
10
21
2
1
```

6 Connecting Inputs and Outputs

Files to stdin

Recall that if we run `grep` without a directory ot file name, it will read from the stdin. For example:

```
grep Duis
```

You can now type the input for `grep` by line line and `grep` will repreat that line if contains the string “Duis”.

Bash provides us a vary handy mechanism that allows us write the contents of a file to the stdin of a process that we start by using the `<` operator. For example we can write the contents of `a_text_file.txt` to the stdin of grep as follows:

```
grep Duis < a_text_file.txt
```

Note that the output will be the same as running:

```
grep Duis a_text_file.txt
```

But with the latter, `grep` opens the file `a_text_file.txt` itself and then read from it, whereas with the former `grep` reads from the stdin and the contents of `a_text_file.txt` are redirected to the stdin.

stdout/stderr to files

We can also redirect the output of a process to a file by using the `>` operator. Let's try it by running:

```
echo hello > test.txt
```

and now inspect the contents of the file:

```
cat test.txt
```

output:

```
hello
```

Let's do it again with a different string:

```
echo world > test.txt
cat test.txt
```

output:

```
world
```

We now see that the previous contents of the file were overwritten by the `>` operator. If we want to append the output of process to a file, we can use the `>>` operator:

```
echo hello > test.txt
echo world >> test.txt
cat test.txt
```

output:

```
hello
world
```

Similar to redirecting the stdout of a process to a file, we can also redirect the stderr using `2>`:

```
ls non_existing_file 2> test.txt
```

Since there is no file or directory with the name `non_existing_file`, the command `ls` printed an error message to the stderr (which we redirected to the file `test.txt`) and terminated. Let's have a look at the error message:

```
cat test.txt
```

output:

```
ls: cannot access 'non_existing_file': No such file or directory
```

Redirecting to the stderr

There is also a way to redirect the stdout of a process to the stderr, which can be done by appending `>&2` to the command. This can be very useful when we are writing a shell script and want to write an error message to the stderr. For example:

```
echo This is an error message >&2
```

The output of a process to the input of another process

The pipe operator `|` redirects the output of the command that is to its left side, to the input of the command that is to its right side. For example

```
cat a_text_file.txt | grep Duis
```

executes `cat` of which output is redirected to `grep`. We can also chain multiple of these redirections after another (that is, we are creating a pipeline):

```
cat a_text_file.txt | grep Duis | sed "s/Duis/siuD/g"
```

executes `cat` which output is redirected to `grep` which output is redirected to `sed`.

Using the output of a process as a value (command substitution)

We can also use the output of a command as a value via command substitution. This is done by using surrounding the command by `$(` and `)`. For example:

```
a=$(echo hello)
```

Assigns the output of `echo` to variable `a`, as we can verify:

```
echo $a
```

output:

```
hello
```

7 Creating a shell script

So far we have been running Bash commands interactively: we type a command, press enter, and immediately see the result. A *shell script* is simply a text file that contains a sequence of commands that Bash can execute for us.

Creating a script file

Let's create a file called `hello.sh`. You can use any text editor. Now type the following contents into the file:

```
#!/usr/bin/bash

echo "Hello from a shell script!"
```

The first line is called the *shebang*. It tells the operating system which program should run this file. Here we use `/usr/bin/bash`, which finds `bash` via your `PATH`.

Making the script executable and running it

By default, a new file is usually not executable. Let's check the permissions:

```
ls -l hello.sh
```

You will likely see that there is no `x` in the permission bits.

We can make the file executable with `chmod`:

```
chmod +x hello.sh
ls -l hello.sh
```

Now we can run the script. Note that we have to specify a path. `./` means "in the current directory":

```
./hello.sh
```

output:

```
Hello from a shell script!
```

If you call `hello.sh` without `./`, Bash will try to find a command with that name in the directories listed in the `PATH` environment variable. Most of the time, the current directory is *not* in `PATH`, so `./` is the normal way to run scripts you just created.

Running a script without execute permissions

Even if a script is not executable, we can still run it by explicitly calling `bash` and passing the script file as an argument:

```
bash hello.sh
```

Script arguments

A script can receive arguments in the same way as a normal command. Inside the script, these arguments are available as `$1`, `$2`, `$3`, ... and the number of arguments is stored in `$#`.

Let's create a script `greet.sh` with the following contents:

```
#!/usr/bin/bash

echo "Script name: $0"
echo "Number of arguments: $#"
echo "Hello $1"
```

Make it executable and run it:

```
chmod +x greet.sh
./greet.sh Alice
```

output:

```
Script name: ./greet.sh
Number of arguments: 1
Hello Alice
```

A small note on comments

In Bash, everything after a `#` (that is not part of the shebang) is treated as a comment and ignored by the shell.

8 Control structures

Now that we can put commands into scripts, we want to make them react to different situations and repeat work. Bash provides control structures such as `if`, loops (`for`, `while`, `until`) and functions.

Exit status and conditions

In Bash, commands indicate success or failure via their *exit status*:

- exit status `0` means success
- any non-zero value means failure

The exit status of the previously executed command is available in `$?`. For example:

```
ls a_text_file.txt
echo $?
ls non_existing_file
echo $?
```

The first `echo $?` will print `0`, while the second will print a non-zero value.

This is important because the `if` statement in Bash does not test "true" or "false" directly. Instead, it runs a command and checks whether that command succeeded.

if / elif / else

The general syntax is:

```
if command-or-condition; then
    commands
elif command-or-condition; then
    commands
else
    commands
fi
```

Using `test` explicitly A very common way to express a condition in Bash is the `test` command. Remember: Bash checks whether a command succeeds (exit status 0). If `test` succeeds, the condition is considered "true".

Some useful tests are:

- integer comparisons: `-eq`, `-ne`, `-lt`, `-le`, `-gt`, `-ge`
- string tests: `=`, `!=`, `-z` (empty), `-n` (non-empty)
- file tests: `-e` (exists), `-f` (regular file), `-d` (directory)

Example with an integer comparison:

```
x=7

if test $x -lt 10; then
    echo "x is smaller than 10"
else
    echo "x is at least 10"
fi
```

Example with a file test:

```
if test -f "some_text_file.txt"; then
    echo "the file exists"
else
    echo "the file does not exist"
fi
```

When you use variables in conditions, it is usually a good idea to put them in quotes to avoid problems with spaces or empty values:

```
name="Alice Bob"

if test -n "$name"; then
    echo "name is non-empty"
fi
```

`[ ... ]` is shorthand for `test` In Bash, `[ ... ]` is just a more convenient syntax for `test`. The following two lines are equivalent:

```
test "$x" -lt 10
[ "$x" -lt 10 ]
```

So we can rewrite the earlier example using `[ ... ]`:

```
x=7

if [ "$x" -lt 10 ]; then
    echo "x is smaller than 10"
else
    echo "x is at least 10"
fi
```

It is important that there is a space after `[` and before `]`:

```
# correct:
[ "$x" -lt 10 ]

# wrong (missing spaces):
["$x" -lt 10]
```

for loops

A `for` loop repeats a block of commands for each element of a list. For example:

```
for n in alice bob charlie; do
    echo "Hello $n"
done
```

Looping over the elements of an array (recall `names=(...)` from the Variables section):

```
names=(alice bob charlie)

for n in ${names[@]}; do
    echo "Hello $n"
done
```

Bash also supports a C-style `for` loop (useful for counting):

```
for ((i=0; i<5; i++)); do
    echo "i=$i"
done
```

while and until loops

A `while` loop runs as long as the condition succeeds (exit status 0). For example, count down from 5:

```
i=5
while [ $i -gt 0 ]; do
    echo $i
    i=$((i - 1))
done
echo "lift off"
```

An `until` loop is the opposite: it runs until the condition succeeds (i.e., it keeps going while the condition fails). For example, count up until we reach 5:

```
i=0
until [ $i -ge 5 ]; do
    echo $i
    i=$((i + 1))
done
```

Both `for` and `while`/`until` loops support:

- `break` to stop the loop immediately
- `continue` to skip to the next iteration

Functions

Functions let us group commands into reusable building blocks. A function is defined as:

```
name() {
    commands
}
```

and called like a normal command: `name arg1 arg2 ...`

Example:

```
greet() {
    local who="$1"
    echo "Hello $who"
}

greet Charlie
greet "Alice Bob"
```

Just like scripts, functions can access their arguments as `$1`, `$2`, ... inside the function. The keyword `local` creates a variable that exists only inside the function (it does not overwrite a variable with the same name outside).

A function can signal success or failure via its return status (again: `0` is success):

```
file_exists() {
    if [ -f "$1" ]; then
        return 0
    else
        return 1
    fi
}

if file_exists "some_text_file.txt"; then
    echo "file exists"
else
    echo "file does not exist"
fi
```

To *output* a value from a function, we typically use `echo` and capture it using command substitution (`$(...)`) as shown earlier:

```
add() {
    echo $(($1 + $2))
}

result=$(add 7 3)
echo $result
```